

ANNOUNCEMENTS

HW1 still accepted through tomorrow night. Project proposals due next Wednesday. Quiz 1 next Monday!

QUIZ 1

In class on Monday. You will have all 65 minutes. Paper exam — bring a pencil! You can bring a one page "cheat sheet"; closed book otherwise. I recommend putting any useful definitions and formulas on there! No code or pseudocode — but make sure you understand how to use each algorithm!

PREPARING FOR THE QUIZ

A test prep doc is available on Canvas, along with a full list of topics and a few practice problems. For each algorithm, make sure you know how to implement it on a small example, when to use it, its main advantages and disadvantages, and any relevant vocabulary (which can go on a cheat sheet). Make sure to read all questions — some will ask you to explain your answer!

QUIZ MATERIAL

The quiz covers material through last class: task environments (types and PEAS), types of agents (reflex vs. model-based vs. goal-based), uninformed search (BFS, DFS, UCS, etc.), informed search (Greedy and A*), heuristic functions (including admissibility), local search (hill climbing, simulated annealing, genetic algorithms), and adversarial search (minimax search, pruning, heuristics).

DEALING WITH UNCERTAINTY

So far, we have been looking at agents with perfect knowledge of the world and how their actions will affect it. In practice, we usually have to deal with some unknown factors. By estimating probabilities of different events, we can make informed choices.

WHY PROBABILITY?

Why use probability instead of trying to create a full model of the world? There are three reasons. First, theoretical ignorance: science does not yet have a perfect model of the real world. Second, practical ignorance: even if we could perfectly model the world, we might not have access to all the relevant information we would need to make a decision. Third, laziness: even if we know all relevant information, perfectly modeling all possible outcomes is a lot of work.

RATIONAL UNCERTAIN AGENTS

If we don't know the consequences of our actions for sure, how can we make rational decisions? The answer is to estimate the probability of different outcomes. This gives us decision theory, which is a combination of probability theory and utility theory. Probability determines how likely we are to wind up in a given state, while utility determines how "good" a given state is. If we have good models for both, we can maximize expected utility.

WHAT IS PROBABILITY?

The probability of an event is the fraction of possible worlds in which that event occurs. The set of all possible worlds is also called the sample space. For example, if I roll a die, there are six possible worlds created by that die roll. I roll a 1 in 1 of those 6 worlds, so $P(\text{roll a 1}) = 1/6$. I roll an even number in 3 of those 6 worlds, so $P(\text{roll even}) = 3/6 = 1/2$.

SOME BASIC RULES OF PROBABILITY

For every event E , $0 \leq P(E) \leq 1$. If an event S happens in every possible world, $P(S) = 1$. If A and B are mutually exclusive events, $P(A \text{ or } B) = P(A) + P(B)$. For any events A and B , $P(A \text{ or } B) = P(A) + P(B) - P(A, B)$, where $P(A, B)$ is the probability that both A and B happen (the joint probability). Finally, $P(\text{not } A) = 1 - P(A)$.

RANDOM VARIABLES

A random variable is a variable that takes on a (possibly) different value for each possible world in the search space. Random variables are usually written with a capitalized first letter. They often (but not always) take on real number values. For example, if we roll two dice, we could have a random variable *Total* whose value is the sum of the two dice values.

PROBABILITY DISTRIBUTIONS

Probability distributions are functions that map the values a random variable could take on to the probabilities that it will take on those values. For example, when rolling two dice, Total is the sum of the values on the two dice and $p(\text{Total})$ is the probability we roll that value. So $p(2) = 1/36$ and $p(7) = 6/36 = 1/6$.

EXPECTED VALUE

We can calculate a random variable's expected value, which is the average of all possible values weighted by their probabilities. For discrete variables, this is the sum of each value multiplied by its probability. Expected value can be used instead of the actual value for random variables in nondeterministic problems.

RECALL: GAME PLAYING

We can use adversarial search to model games. The basic idea is to model out future moves as a search tree, assuming that all players always make the optimal move. So far, we have assumed no randomness — we can perfectly predict the outcome of all of our moves.

RECALL: IMPROVING SEARCH

Alpha-beta pruning prunes paths that correspond to guaranteed suboptimal moves. Heuristic minimax treats some non-terminal states as goal states by using an evaluation function to get their utility. Forward pruning artificially limits the branching factor by only looking at "good" moves. All of these rely on no randomness — they prune away moves that are "guaranteed" bad based on either the tree structure or the evaluation function.

STOCHASTIC GAMES

Stochastic games assume two-player, perfect information, turn-taking games but allow for randomness. An example is backgammon. At all times, there are three options: Max makes a choice, Min makes a choice, or something random happens.

EXPECTIMINIMAX

We can do minimax as before for stochastic games, but we need to add chance nodes that account for randomness. A chance node's minimax value is the expected utility based on its children, so we need to know the probability of each child. Otherwise it is the same recursive algorithm as minimax.

Pseudocode:

Expectiminimax(node): if node is terminal: return utility of node else: calculate Minimax(child) for each child of node if to-move is Max: return maximum value of Expectiminimax(child) if to-move is Min: return minimum value of Expectiminimax(child) if to-move is Chance: return sum over children of $P(\text{child}) * \text{Expectiminimax}(\text{child})$

EXAMPLE: COIN FLIPPING GAME

Two weighted coins: coin A flips heads with a chance of 40%, coin B with a chance of 75%. Each player picks one of the two coins and flips it (Max goes first). If the coin flips are different, Max wins. If the coin flips are the same, Min wins.

The expectiminimax tree is built with Max at the root choosing between coins A and B. Each choice leads to a chance node based on the flip outcome, then Min chooses a coin, and so on down the tree. Computing expected values at each chance node and propagating up yields the optimal strategy.

HEURISTIC EXPECTIMINIMAX

We can (and often will) use heuristic expectiminimax. Evaluation functions should match expected utility as often as possible. If evaluation functions are not proportional to utility, calculations at chance nodes can lead to wrong choices — and this can occur even if better moves have higher scores, unlike in normal minimax.

EXPECTIMINIMAX COMPLEXITY

Minimax complexity is $O(b^m)$. Expectiminimax must also consider n , the number of distinct outcomes from each chance node, which effectively acts as a second branching factor. The complexity of expectiminimax is therefore $O(b^m * n^m)$.

IMPROVING EXPECTIMINIMAX

Reducing tree size is both more important and more difficult in stochastic games than in deterministic ones. The same approaches still apply. Alpha-beta pruning works the same and is very common. Heuristic minimax also applies similarly. Forward pruning can focus on only "good" moves for Max/Min nodes, but not for chance nodes, so aggressive pruning is often necessary to keep the overall branching factor low.

WRAPPING UP

Probability is used to make rational decisions in nondeterministic environments. We can adapt minimax search to stochastic games by adding chance nodes that are computed as the expected value of their children. Expectiminimax allows us to handle more games, but also introduces larger trees and a greater need for pruning. We will see other probability-based approaches in future classes.

Naïve Bayes Rose Sloan r.sloan@northeastern.edu

RECALL: CONDITIONAL PROBABILITY

The conditional probability $P(A|B)$ is the probability of A given B. It allows us to determine the probability of unobservable events given observable results — for example, what is the probability someone has a disease given certain lab test results? We can use this to make inferences and solve problems.

INDEPENDENCE

We say that events A and B are independent if $P(A|B) = P(A)$. Because of Bayes' Theorem, which states $P(A|B)P(B) = P(A,B)$, if A and B are independent then $P(A,B) = P(A)P(B)$. For example, $P(\text{person is left-handed}) = 10\%$, $P(\text{person is female}) = 50\%$, $P(\text{person is left-handed and female}) = 5\%$, and $P(\text{person is left-handed} \mid \text{person is female}) = 10\%$.

CONDITIONAL INDEPENDENCE

Two events A and B are independent if $P(A|B) = P(A)$ or $P(A)P(B) = P(A,B)$. We say that two events A and B are conditionally independent given C if $P(A|C)P(B|C) = P(A,B|C)$. Note that conditional independence does not imply independence, and vice versa.

Example 1: Coin 1 is a fair coin. Coin 2 is an unfair coin that only lands on heads 1/4 of the time. We randomly pick between the two coins, then flip that coin twice. Let A be the event that our first flip lands on heads, B that our second flip lands on heads, and C that we pick coin 1. Then

$P(A) = P(B) = 1/2 * 1/2 + 1/2 * 1/4 = 3/8$, and $P(A,B) = 1/2(1/2 * 1/2) + 1/2(1/4 * 1/4) = 5/32$, which does not equal $P(A)P(B)$. So A and B are not independent. However, $P(A|C) = 1/2$, $P(B|C) = 1/2$, and $P(A,B|C) = 1/2 * 1/2 = 1/4$. Since $P(A|C)P(B|C) = P(A,B|C)$, they are conditionally independent — even though they are not independent!

Example 2: Flip a fair coin twice. Let A be the event that the first flip is heads, B that the second flip is heads, and C that both flips are the same. Clearly A and B are independent: $P(A) = P(B) = 1/2$ and $P(A,B) = 1/4$. However, A and B are not conditionally independent given C: $P(A|C) = P(B|C) = 1/2$, but $P(A,B|C) = 1/2$. Therefore A and B are independent but not conditionally independent.

CLASSIFICATION PROBLEMS

The basic idea is: given information about the current state, sort it into one of a pre-set group of labels. This may be used to decide on the next move or simply to classify the state. The set of labels is discrete and determined as part of the problem formulation, and every state belongs to exactly one class. This is usually approached using supervised machine learning, where we look at a set of known correct state-label pairs and build a model that can predict labels for unseen states.

Example classification problems include spam filtering, medical diagnosis, sentiment or emotion analysis of text, handwriting recognition of single characters, predicting the best next chess move, plant species classification, and predicting the topic of an article.

BAYES' THEOREM FOR CLASSIFICATION

Bayes' Theorem lets us predict the probability of an unobservable event based on observable evidence. Classification problems require us to figure out what class (not directly observable) produced some directly observable state. In theory, if we can figure out $P(\text{label}|\text{state})$ for each label, we can determine the most likely label for a given state. For example, if we are classifying what season an image was taken in and we see red leaves and pumpkins, $P(\text{fall}|\text{red leaves, pumpkins})$ is probably higher than $P(\text{summer}|\text{red leaves, pumpkins})$.

In a concrete example, suppose we want to predict whether someone plays tennis given certain weather conditions. The probabilistic question is: given that it is sunny, hot, humid, and windy, is $P(\text{playTennis}|\text{sunny, hot, humid, windy})$ greater than $P(\text{don't play tennis}|\text{sunny, hot, humid, windy})$? For binary classification, we can just check if the probability of playing tennis exceeds 50%.

SUPERVISED LEARNING BASICS

Supervised learning works by using a large amount of training data to learn a model — in this case, by estimating various probabilities. The training set contains a (usually large) number of previously observed state-label pairs. We also typically have a test set of state-label pairs that we do not use to build the model, but use to evaluate how accurately the learned model classifies new states. We should not use or even look at the test set until after the model has been trained.

NAÏVE BAYES

Naïve Bayes is a classification method where we return the class label with the highest conditional probability given the input features. Formally, given input features $x_0, x_1, \dots, x_n$ and possible class labels $C_0, C_1, \dots, C_m$, we return the class C_k that maximizes $P(C_k | x_0, x_1, \dots, x_n)$. To calculate these probabilities, we assume that all features are conditionally independent given all class labels — formally, for any i, j, k with $i \neq j$, $P(x_i, x_j | C_k) = P(x_i | C_k)P(x_j | C_k)$. Note that this assumption is usually not actually true.

CONDITIONAL INDEPENDENCE PROPERTIES

We have defined A and B as conditionally independent given C if $P(A, B | C) = P(A | C)P(B | C)$. This is equivalent to saying $P(A | B, C) = P(A | C)$.

The chain rule for probability (not to be confused with the calculus chain rule) states that for any events A, B , and C : $P(A, B, C) = P(A | B, C)P(B | C)P(C)$. This extends to any n events, and if all other events are conditionally independent given the last, the joint probability simplifies to a product of individual conditional probabilities.

NAÏVE BAYES: FINDING THE PROBABILITY

For each class C_k , we must find $P(C_k | x_1, x_2, \dots, x_n)$. Using the definition of conditional probability and the conditional independence assumption, this simplifies to a product of individual likelihoods times the prior: $P(C_k) * P(x_1 | C_k) * P(x_2 | C_k) * \dots * P(x_n | C_k)$.

Since $P(x)$ remains the same across different classes, we can ignore it when comparing classes — we are effectively comparing joint probabilities rather than conditional probabilities. The prior $P(C_k)$ is the fraction of training data points belonging to class C_k , and $P(x_i | C_k)$ is the fraction of training data points in class C_k that have feature x_i .

EXAMPLE: PLAYING TENNIS

Training data (14 days with features Weather, Temperature, Humidity, Wind, and label Play Tennis):

Day 1: Sunny, Hot, High, Weak — No Day 2: Sunny, Hot, High, Strong — No Day 3: Overcast, Hot, High, Weak — Yes Day 4: Rain, Mild, High, Weak — Yes Day 5: Rain, Cool, Normal, Weak — Yes Day 6: Rain, Cool, Normal, Strong — No Day 7: Overcast, Cool, Normal, Strong — Yes Day 8: Sunny, Mild, High, Weak — No Day 9: Sunny, Cool, Normal, Weak — Yes Day 10: Rain, Mild, Normal, Weak — Yes Day 11: Sunny, Mild, Normal, Strong — Yes Day 12: Overcast, Mild, High, Strong — Yes Day 13: Overcast, Hot, Normal, Weak — Yes Day 14: Rain, Mild, High, Strong — No

To classify a day that is Sunny, Hot, Humid, and Windy, we compare $P(\text{Yes, sunny, hot, humid, windy})$ and $P(\text{No, sunny, hot, humid, windy})$.

Priors: $P(\text{Yes}) = 9/14$, $P(\text{No}) = 5/14$.

Likelihoods from training data: $P(\text{sunny}|\text{Yes}) = 2/9$, $P(\text{sunny}|\text{No}) = 3/5$ $P(\text{hot}|\text{Yes}) = 2/9$, $P(\text{hot}|\text{No}) = 2/5$ $P(\text{humid}|\text{Yes}) = 3/9$, $P(\text{humid}|\text{No}) = 4/5$ $P(\text{windy}|\text{Yes}) = 3/9$, $P(\text{windy}|\text{No}) = 3/5$

Multiplying these together with the priors, $P(\text{No, sunny, hot, humid, windy})$ comes out higher, so we predict No.

FINDING $P(x)$

In some cases we may wish to know the actual probability $P(C_k|x)$. To do this we must calculate $P(x)$. Note that x_i and x_j are not independent (only conditionally independent), so we cannot simply multiply their marginal probabilities. However, since x must belong to some class, we can sum over all classes: $P(x) = \sum \text{over all } C_k \text{ of } P(x|C_k)P(C_k)$. This lets us normalize the joint probabilities into true conditional probabilities.

TEXT CLASSIFICATION

Text classification involves sorting an entire document into one of a set of discrete categories. Problems in this area include spam detection, genre classification, language identification, authorship analysis, and sentiment analysis. We will focus on sentiment analysis.

Sentiment analysis in its simplest form determines whether a document primarily expresses positive or negative feelings, sometimes also allowing a neutral classification. This is also known as determining polarity. Applications include analyzing product or movie reviews, sorting customer feedback, and gauging public opinion via social media.

BAYES' THEOREM FOR TEXT CLASSIFICATION

For sentiment analysis we need a labeled training corpus of documents marked positive or negative. For a new document we calculate $P(+|\text{document})$ and $P(-|\text{document})$. Priors are simple — divide the number of documents with a given label by the total number of documents. Likelihoods are harder: what is $P(\text{"I loved this movie"}|+)$?

BAG OF WORDS MODEL

The bag of words model treats all documents as a collection of words, ignoring word order and looking only at which words appear and how many times. This works because positive documents tend to contain many positive words. However, losing word order does lose important information — for example, "not bad" becomes just "not" and "bad", both of which are negative on their own.

NAÏVE BAYES ASSUMPTION FOR BAG OF WORDS

To calculate likelihoods, we make the naïve Bayes assumption that all words are conditionally independent given the class. Then $P(w_1, w_2, \dots, w_n|c) = P(w_1|c)P(w_2|c)\dots P(w_n|c)$. For example, $P(\text{"Loved this movie"}|+) = P(\text{loved}|+)P(\text{this}|+)P(\text{movie}|+)$. The assumption applies even with repeated words: $P(\text{"Really really good"}|+) = P(\text{really}|+)P(\text{really}|+)P(\text{good}|+)$. We then find the class c that maximizes $P(w_1|c)P(w_2|c)\dots P(w_n|c)P(c)$.

ESTIMATING WORD PROBABILITIES

For every word w and each class c , we estimate $P(w|c)$ as the number of instances of w in class c divided by the total number of words in class c . For example, if all positive documents contain 1000 words total and "good" appears 20 times among them, then $P(\text{good}|+) = 20/1000 = 0.02$.

EXAMPLE: MOVIE REVIEWS

Training set: "Good cheesy fun" — positive "Really entertaining movie" — positive "Surprisingly good movie" — positive "Watch a good movie instead" — negative "Not fun or worth your time" — negative

Test: "Good fun movie"

$P(+)$ = 3/5, $P(-)$ = 2/5. There are 9 words in positive reviews and 11 in negative reviews.

$P(\text{"good"}|+) = 2/9$, $P(\text{"good"}|-) = 1/11$ $P(\text{"fun"}|+) = 1/9$, $P(\text{"fun"}|-) = 1/11$ $P(\text{"movie"}|+) = 2/9$,
 $P(\text{"movie"}|-) = 1/11$

Computing the joint probabilities, the positive class probability (approximately 0.0033) is about 10 times larger than the negative class probability (approximately 0.0003), so we classify the review as positive.

MISSING VALUES AND SMOOTHING

One problem with this approach is what happens if a word in the test set never appears in the training set. This is fairly likely to happen, since natural languages have hundreds of thousands of words, new words are coined frequently, and documents may contain typos. If a word w never appears in training data for class C_k , then $P(w|C_k) = 0$, which makes $P(C_k|x) = 0$ for any document containing w , regardless of all other words.

As a principle, we should not assume something is impossible just because it did not appear in training data.

LaPlace smoothing addresses this by assuming there is one additional occurrence of each word in our vocabulary added to every class. If V is the vocabulary size, then:

$$P(w|C_k) = (\text{count of } w \text{ in class } C_k + 1) / (\text{total words in class } C_k + V)$$

Applying this to the movie review example with a vocabulary of 15 unique words: $P(\text{"good"}|+) = 3/24$, $P(\text{"good"}|-) = 2/26$ $P(\text{"fun"}|+) = 2/24$, $P(\text{"fun"}|-) = 2/26$ $P(\text{"movie"}|+) = 3/24$, $P(\text{"movie"}|-) = 2/26$

After normalizing, there is an 81.1% chance the review is positive.

NAÏVE BAYES CONCLUSIONS

Naïve Bayes is a classification method built around using Bayesian probability to determine the probability of a class label given a set of features. It relies on the assumption that features are conditionally independent of each other, which is usually not true but is close enough to work well in practice. Naïve Bayes is often a strong approach for text classification, and smoothing is important to avoid zero probabilities.

Intro to Language Modeling Rose Sloan r.sloan@northeastern.edu

ANNOUNCEMENTS

Upcoming deadlines: Lab 1 released late this week or early next week. Quiz 2 is February 23. Literature review is due February 26.

LITERATURE REVIEW

The literature review is due in 2 weeks. You must find a relevant academic paper, summarize it, and explain its relevance to your project.

What counts as a valid source: journal articles and conference papers are good. Books, student theses, websites, datasets without an associated paper, and pre-prints (such as those on arXiv) do not count.

Where to find papers: Google Scholar, library databases, IEEExplore, ACM Digital Library, conference proceedings, and other papers' bibliographies. Always sign in for institutional access!

Tips for reading papers: always start with the abstract. It is often helpful to look at major graphs and tables before reading the full paper. Give yourself time, and look for specific connections to your project.

A warning about AI: LLMs can be useful for summarization and rewording, but they can misinterpret or oversimplify academic and technical writing, so always double-check. Do not use AI to find papers — they will make up fake ones, and you cannot get an A on the final paper if you have fake citations. Targeted tools like Zotero are usually better for bibliography creation.

BEYOND BAG OF WORDS

Last class we used Naïve Bayes to classify documents based on their words. This is a good simple approach because documents belonging to a category tend to have lots of relevant words. But what happens with tasks where word order matters? This is especially important when the task involves language generation.

LANGUAGE MODELING MOTIVATION

We know intuitively which sentences are likely or unlikely. Many factors are at play: grammaticality, meaning, real-world context, tone, and vocabulary choice. We would like to teach the computer some of these insights.

Language modeling has many uses. Many NLP tasks involve outputting text, such as dialogue systems, machine translation, speech recognition, and spelling or grammar correction. If we know how likely or unlikely sentences are, we can narrow down possible outputs and hopefully

produce reasonable answers. For example, "John made his bed" is far more likely than "John ate his bed."

To illustrate: given the prompt "I just had a sandwich for ____", a likely completion is "lunch," a surprising one is "breakfast," an unlikely one is "fun," and an ungrammatical one would be "yellow."

USING A CORPUS

To find out what sentences are likely, we look at what sentences appear in the wild. A corpus is a collection of text and/or speech used for linguistic analysis or modeling. It may be scraped from the internet or other media, or collected for research purposes. It may be very general (all English-language data on the internet) or very specific (a collection of tweets tagged #sarcasm).

LANGUAGE MODELING, FORMALLY

For any sequence of words $w_1, w_2, \dots, w_n$, we want to find the probability that those words appear in that order. We want to learn these probabilities from a corpus, usually for a single language using a monolingual corpus.

A naive approach would be to count how many times a given sequence appears in the corpus and divide by the total number of n -word strings. The problem is that this does not account for new sentences.

TOKENIZATION

Rather than treating a block of text as a single unit, we convert it into a list of tokens through a process called tokenization. Each word is a token, as is punctuation, numbers, and other symbols. Tokenization is relatively easy in English but harder in some other languages.

Example: "It was the best of times, It was the worst of times" becomes ['It', 'was', 'the', 'best', 'of', 'times', ',', 'it', 'was', 'the', 'worst', 'of', 'times', '.']

THE UNIGRAM MODEL

The simplest approach is to look only at how common each individual word is. We get individual word probabilities by counting how frequently a word appears in the corpus. For example, in a sample corpus: $P(\text{the}) \approx 5.8\%$, $P(\text{cat}) \approx 0.03\%$, $P(\text{sat}) \approx 0.06\%$, $P(\text{by}) \approx 0.33\%$, $P(\text{window}) \approx$

0.02%. We then multiply all the individual word probabilities together to get the sentence probability.

The shortcoming of this approach is that while unigram probabilities are easy to calculate, they make very strong independence assumptions and ignore word order entirely. Under this model, $P(\text{"The cat sat by the window"}) = P(\text{"Sat the the window cat by"})$. It also ignores context — the word "lieu" is uncommon in general but much more common in the context "in ___ of."

LANGUAGE MODELS AND CONDITIONAL PROBABILITY

We want to find the probability of a sequence of words. Using the chain rule of probability, this becomes a product of conditional probabilities: the probability of each word given all the words before it. A language model works as long as we can calculate or approximate $P(w_k | w_1, \dots, w_{k-1})$ for any position k — colloquially, the probability of a word given all previous words in the sentence.

N-GRAM MODELS

The unigram model approximates $P(w_k | w_1, \dots, w_{k-1})$ as simply $P(w_k)$, ignoring all context. The bigram model improves on this by using one word of context, approximating $P(w_k | w_1, \dots, w_{k-1})$ as $P(w_k | w_{k-1})$. Bigram probabilities are calculated by counting the frequency of the two-word string " $w_{k-1} w_k$ " and dividing by the frequency of w_{k-1} . For example, "the" appears 1679 times and "the cat" appears 2 times, so $P(\text{cat}|\text{the}) = 2/1679 \approx 0.12\%$.

The trigram model goes further by looking at two words of context. For example, "by the" appears 27 times and "by the way" appears 2 times, so $P(\text{way}|\text{by the}) \approx 7.4\%$. Trigrams are more likely to be missing from the corpus, but trigram models are often reasonable for generation.

In theory we can use n -grams for any n , but in practice, above $n = 3$ or 4 , data sparsity becomes a serious issue. Many 5-grams in new sentences will not have appeared in the corpus at all. Generating from 4-gram or 5-gram models will often simply reproduce sentences from the corpus verbatim.

IMPLEMENTATION: STARTS AND ENDS

We usually model one sentence at a time and need to account for the fact that some words are more or less common at the starts and ends of sentences. For example, "and" is very common overall but usually appears in the middle of sentences. The solution is to pad all sentences with $n-1$ START tokens and one END token.

Example: "The cat sat there" becomes [START, the, cat, sat, there, END], and the sentence probability is computed as $P(\text{the}|\text{START}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{cat}) * P(\text{there}|\text{sat}) * P(\text{END}|\text{there})$.

SAMPLING

A useful way to understand what a language model is doing is to sample sentences from it. Sampling from a bigram model works as follows: start by randomly choosing a bigram (START, w) based on probability, then choose a random bigram (w, x), and repeat until a bigram ending with the END token is chosen.

Examples trained on Alice in Wonderland:

Unigrams: "not you for mine, and a to about said the her her picking in think it" Bigrams: "what would change the branches and was always six is i should have you mean what" Trigrams: "and so it was not a regular rule you invented it just at first but she" 4-grams: "alice had never been so much contradicted in her life before and she felt certain it"

As n increases, the output becomes noticeably more coherent.

A NOTE ON ZERO VALUES

For any n-gram model we will encounter zero values when a test sentence contains a substring not present in the corpus. Since there are always new ways to form sentences, this is inevitable for anything beyond unigrams. There are a few solutions. Smoothing applies add-one smoothing similar to what we used in Naïve Bayes, though this is not the most effective approach. Interpolation takes a weighted average of the unigram, bigram, and trigram probabilities. Backoff uses whichever of trigrams, bigrams, or unigrams gives the largest nonzero value.

N-GRAMS: PROS AND CONS

On the positive side, n-gram models are easy to train and use, can work with small or large amounts of data, provide enough insight about word order and word choices to be useful as part of larger applications, and are often "good enough" when part of a bigger system.

On the negative side, they have no way to track "similar" words that appear in similar contexts — for example, learning that if "lunch" is common in a certain context, "dinner" might be too. They also cannot use more than one or two words of context, which means generated text is likely to be incoherent and may simply repeat large segments of the corpus.

WRAPPING UP

Language modeling allows us to predict how "likely" a piece of text is, which is useful for any task that generates text. It is based on calculating the conditional probability of a word given its context — this is how all language models work. N-grams simplify this by looking at only one or two words of context. They are useful when combined with other models, but are very limited on their own.

Markov Models

Instructor: Rose Sloan (r.sloan@northeastern.edu)

Dealing with Time

- Have often dealt with problems where we care about sequences of events/states
 - Useful for language applications, but many other applications
 - Nondeterministic games, video processing, weather, stock values...
 - Would like to figure out the probabilities of different sequences
-

Markov Models

- General way of keeping track of series of events
 - Each event will be an atomic state
 - Each transition between states has a probability
 - Can keep track of longer paths of states
-

The Markov Assumption

- Primary assumption: the probability of being in a given state depends only on the previous state
 - This is the first-order Markov assumption, can have versions of Markov chains that look further back
 - Additional assumption: transition probabilities are time-homogeneous—do not change over time
-

Markov Chains: Formally

- Markov chains have three components:
 - A set of N states $q_1, q_2, \dots, q_N$
 - A transition matrix A such that $A_{\{i,j\}}$ is the probability that we move from state q_i to state q_j
 - For all i , the sum of $A_{\{i,1\}}$ through $A_{\{i,N\}}$ must be 1
 - An initial probability distribution π_0 that specifies the probability we start in q_i for each i
 - Sum of all starting probabilities must also be 1
-

Probabilities of Sequences

- Note: $A_{\{i,j\}}$ is just $P(q_j|q_i)$
 - To find the probability of a sequence, we can apply chain rule of probability
-

A Markov Chain: Weather

(Diagram reference)

Using the Chain

- Let's say starting probability of sun is 0.4
 - What is $P(\text{sun, sun, sun})$?
 - $P(\text{sun, sun, sun}) = P(\text{sun})P(\text{sun}|\text{sun})P(\text{sun}|\text{sun}) = 0.4 * 0.7 * 0.7 = 0.196$
-

Using the Chain

- Starting probability of sun is 0.4
- What is $P(\text{sun, cloud, rain})$?

PollEv.com/roslloan768

Using the Chain

- Starting probability of sun is 0.4
 - What is $P(\text{sun, cloud, rain})$?
 - $P(\text{sun, cloud, rain}) = P(\text{sun})P(\text{cloud}|\text{sun})P(\text{rain}|\text{cloud}) = 0.4 * 0.2 * 0.4 = 0.032$
-

Probabilities Over Time

- Starting probabilities of sun, cloud, and rain are 0.4, 0.4, and 0.2
 - What is the probability of sun on the second day?
 - $P(\text{sun, sun}) + P(\text{cloud, sun}) + P(\text{rain, sun}) = P(\text{sun})P(\text{sun}|\text{sun}) + P(\text{cloud})P(\text{sun}|\text{cloud}) + P(\text{rain})P(\text{sun}|\text{rain}) = 0.4 * 0.7 + 0.4 * 0.2 + 0.2 * 0.33 = 0.426$
-

Probabilities Over Time

- What is the probability of sun on the second day?
 - $P(\text{sun})P(\text{sun}|\text{sun}) + P(\text{cloud})P(\text{sun}|\text{cloud}) + P(\text{rain})P(\text{sun}|\text{rain})$
 - Two observations:
 - Probabilities of states change over time (even though transition probabilities do not)
 - Probability on day 2 is the dot product of the starting probabilities and the transition matrix column pointing towards sun
-

Probabilities

- Recall: we can write the starting probabilities as a vector π_0 and the transition probabilities as a matrix A
 - Can get probabilities of each state on second day (π_1) by simply multiplying these
 - Can also do this at future time steps:
-

A Stable State

- We know $\pi_{t+1} = \pi_t$ if and only if $\pi_t A = \pi_t$
- If this is true, the probabilities will not change at any future point!
- Mathematically, given a matrix A and nonzero vector v , we call v an eigenvector if there is a scalar λ such that: $Av = \lambda v$

- Here, π_t must be an eigenvector with a corresponding eigenvalue of 1
 - Can figure out a Markov chain's stable state from this!
-

Thinking Over Time

- Can easily determine probabilities of consecutive events
 - Can determine overall probabilities of states in the long term
 - What about the probabilities over medium amounts of time?
 - E.g. If it is sunny today, what is the probability it is rainy in one week?
-

Thinking ahead

- If it is sunny today, what is the probability it is rainy tomorrow?
 - $P(\text{rain}|\text{sun}) = 0.1$
 - If it is sunny today, what is the probability it is rainy in two days?
 - Must account for all intermediate days
 - $P(\text{rain}|\text{sun})P(\text{sun}|\text{sun}) + P(\text{rain}|\text{cloud})P(\text{cloud}|\text{sun}) + P(\text{rain}|\text{rain})P(\text{rain}|\text{sun})$
 - Note that this is just the dot product of the row of transitions from rain with the column of transitions to sun—which is an entry in A^2
-

Kolmogorov-Chapman Equation

- In general, probability of transitioning from q_i to q_j in 2 steps is $(A^2)_{\{i,j\}}$
 - Can extend same logic to show probability of transitioning in n steps is $(A^n)_{\{i,j\}}$
-

In the Long Term

- What happens to A^n as n goes to ∞ ?
 - Over time, each row becomes the stable state of π
 - This means that starting probabilities do not matter long term...unless some states are unreachable from other states
-

The Gambler's Ruin

- Consider a gambler who plays a betting game with even odds
 - 50% chance of winning \$25, 50% of losing \$25
- Quit if all money is gone or if we have \$100
- What happens if we start with \$50?

Gambler's Ruin

(Series of diagrams showing the Markov chain)

Gambler's Ruin

- Eventual state probabilities are clearly based on starting state
 - Starting with \$25: 25% chance of winning
 - Starting with \$50: 50% chance of winning
 - Starting with \$75: 75% chance of winning
-

A Note on Finding Probabilities

- Note: all these methods rely on us knowing the transition probabilities up front
 - This is not always the case
 - May need to simulate many possible sequences of events and estimate probabilities from simulations
 - These are called Monte Carlo methods
-

Hidden States

- So far, have been looking at cases with sequences of observable states
 - However, we may be in situations where the states are not directly observable
 - Similar to what we used Bayes Theorem for
 - Would like a version of Markov models that consider both transitions between states, and how states correspond to observable events
-

Hidden Markov Models

- Hidden Markov models are used to model sequences where the states are not directly observable
 - Each state will emit an observation, leading to an observable sequence
 - Each state will have a set of emission probabilities that determine which observations it is likely to emit
 - We assume that the probability of an observation is dependent only on which state emitted it
-

HMMs: Formally

- A set of N hidden states $q_1, q_2, \dots, q_N$
 - A transition matrix A such that $A_{i,j}$ is the probability that we move from state q_i to state q_j
 - An initial probability distribution π such that π_i is the probability our sequence starts with q_i
 - A set of O possible observations $o_1, o_2, \dots, o_O$
 - An emission probability $b_i(o_j)$ for each $i \leq N$ and $j \leq O$, indicating the probability that state q_i emits o_j
 - Sum of $b_i(o_1)$ through $b_i(o_O)$
-

Example: Train Timing

- Determine whether my train was late (over a series of days), based on my emotions
- Train lateness is unobservable, mood is observable

Example: Train Timing

(Diagram reference)

HMM Tasks

- Working with HMMs involves three tasks
 - Likelihood: determine the probability of a sequence of observations
 - Decoding: given a sequence of observations, determine the most likely sequence of states that caused them
 - Learning: learn the parameters of an HMM

Likelihood Calculations

- Calculating series of states is the same as before
- If starting probability of Late is 0.4, what is $P(\text{Late}, \text{Late})$?
- $P(\text{Late}, \text{Late}) = P(\text{Late})P(\text{Late}|\text{Late}) = 0.4 \cdot 0.2 = 0.08$

Likelihood Calculations

- If starting probability of Late is 0.4, what is $P(\text{Late} \ \& \ \text{Happy}, \text{Late} \ \& \ \text{Sad})$?
- Now must add in emission probabilities!
- $P(\text{Late} \ \& \ \text{Happy}, \text{Late} \ \& \ \text{Sad}) = P(\text{Late})P(\text{Happy}|\text{Late})P(\text{Late}|\text{Late})P(\text{Sad}|\text{Late}) = 0.4 \cdot 0.5 \cdot 0.2 \cdot 0.5 = 0.02$

Likelihood Calculations

- Given starting probabilities, what is $P(\text{Happy}, \text{Sad}, \text{Happy})$?
- Now must sum over every possible sequence of states!

The Forward Algorithm

- Let's use a dynamic programming approach instead
- Work forward in time, keep track of the probabilities of different hidden states we could be in
- This is called the forward trellis
- Define $\alpha_t(q_i)$ as the probability we're in state q_i at time t —and that the given sequence of observations has occurred
- At the end, can just sum of over α_T of all states!

Calculating α

- At the start, α_1 is just given by starting probabilities multiplied by emission probabilities
 - E.g. $\alpha_1(\text{Late}) = P(\text{Late}) * P(\text{Happy}|\text{Late})$

- Otherwise, recurse:

Forward Algorithm: Pseudocode

```

for all i from 1 through N:
    initialize  $\alpha_1(q_i) = P(q_i)P(o_1|q_i)$ 
for all t from 2 through T:
    for all i from 1 through N:
        set  $\alpha_t(q_i)$  to the sum over all j of  $\alpha_{t-1}(q_j) P(q_i|q_j)P(o_t|q_i)$ 
return sum over all i of  $\alpha_T(q_i)$ 

```

Forward Algorithm Example

(Series of diagrams showing the forward algorithm trellis with Happy, Sad, Happy observations and OT, L, VL states)

Forward Algorithm Example

H S H

OT: $P(OT)P(H|OT) = 0.36$

L: $P(L)P(H|L) = 0.2$

VL: $P(VL)P(H|VL) = 0.08$

Forward Algorithm Example

OT: $0.36 \rightarrow *P(OT|OT)=0.8, *P(S|OT)=0.1$

L: $0.2 \rightarrow *P(OT|L)=0.4$

VL: $0.08 \rightarrow *P(OT|VL)=0.6$

OT: $0.1*(0.36*0.8 + 0.2*0.4 + 0.08*0.6) = 0.0416$

Forward Algorithm Example

OT: 0.0416

L: 0.05

VL: 0.0744

Forward Algorithm Example

OT: $0.9 \cdot (0.0416 \cdot 0.8 + 0.05 \cdot 0.4 + 0.0744 \cdot 0.6) = 0.0881$

L: 0.0182

VL: 0.0126

$P(\text{happy, sad, happy}) = 0.0881 + 0.0182 + 0.0126 = 0.119$

Decoding

- Forward algorithm tells us probability of observation sequence, regardless of underlying states
 - May want to know what states could have produced a given sequence
 - This is a task called decoding
 - Will use a dynamic programming approach again
-

The Viterbi Algorithm

- Dynamic programming approach to decoding a sequence of observations using an HMM
 - Again, work forward, looking at each possible state at each time step
 - We can use the same lattice approach
 - Define $v_t(q_i)$ as the probability that we're in state q_i at time t , assuming that we took the most probable path there
-

Calculating v

- At $t=1$, identical to forward algorithm
 - $v_1(q_i) = P(q_i) \cdot P(o_1|q_i)$
 - For all future time steps, look at work recursively—this time finding the previous state that maximizes the probability
 - $v_t(q_i) = \max_j (v_{t-1}(q_j) P(q_i|q_j) P(o_t|q_i))$
-

Pseudocode: Viterbi Algorithm

for all i from 1 through N :

```

initialize  $v_1(q_i) = P(q_i)P(o_1|q_i)$ 
for all t from 2 through T:
  for all i from 1 through N:
    set  $v_t(q_i)$  to the maximum of all j of  $v_{t-1}(q_j) P(q_i|q_j)P(o_t|q_i)$ 
    create a backpointer from  $v_t(q_i)$  to the argmax
return the path traced back from the largest value of  $v_T$ 

```

Viterbi Algorithm Example

(Series of diagrams showing the Viterbi algorithm trellis)

Viterbi Algorithm Example

H S H

OT: $P(OT)P(H|OT) = 0.36$

L: $P(L)P(H|L) = 0.2$

VL: $P(VL)P(H|VL) = 0.08$

Viterbi Algorithm Example

OT: 0.36

L: 0.2

VL: 0.08

OT: $\max((0.36*0.8*0.1), (0.2*0.4*0.1), (0.08*0.6*0.1)) = 0.0288$

Viterbi Algorithm Example

OT: 0.0288

L: 0.02

VL: 0.048

Viterbi Algorithm Example

OT: $\max((0.0288*0.8*0.9), (0.02*0.4*0.9), (0.048*0.6*0.9)) = 0.0259$

L: 0.0072

VL: 0.0032

Hidden Markov Models

Instructor: Rose Sloan (r.sloan@northeastern.edu)

Recall: Markov Models

- Used to track probabilities of sequences
 - Based on the first order Markov assumption: current state's probability is only dependent on the previous step
 - Introduced hidden Markov models (HMMs) for situations where states are not directly observable
 - HMMs include emission probabilities that each state will produce a given observation
-

Example: Train Timing

- Determine whether my train was late (over a series of days), based on my emotions
- Train lateness is unobservable, mood is observable

Example: Train Timing

(Diagram reference)

HMM Tasks

- Working with HMMs involves three tasks
 - Likelihood: determine the probability of a sequence of observations
 - Decoding: given a sequence of observations, determine the most likely sequence of states that caused them
 - Learning: learn the parameters of an HMM
-

Likelihood Calculations

- Calculating series of states is the same as before
- If starting probability of Late is 0.4, what is $P(\text{Late}, \text{Late})$?

- $P(\text{Late}, \text{Late}) = P(\text{Late})P(\text{Late}|\text{Late}) = 0.40.2 = 0.08$
-

Likelihood Calculations

- If starting probability of Late is 0.4, what is $P(\text{Late} \ \& \ \text{Happy}, \text{Late}\&\text{Sad})$?
 - Now must add in emission probabilities!
 - $P(\text{Late} \ \& \ \text{Happy}, \text{Late}\&\text{Sad}) = P(\text{Late})P(\text{Happy}|\text{Late})P(\text{Late}|\text{Late})P(\text{Sad}|\text{Late}) = 0.40.50.20.5 = 0.02$
-

Likelihood Calculations

- Given starting probabilities, what is $P(\text{Happy}, \text{Sad}, \text{Happy})$?
 - Now must sum over every possible sequence of states!
-

The Forward Algorithm

- Let's use a dynamic programming approach instead
 - Work forward in time, keep track of the probabilities of different hidden states we could be in
 - This is called the forward trellis
 - Define $\alpha_t(q_i)$ as the probability we're in state q_i at time t —and that the given sequence of observations has occurred
 - At the end, can just sum of over α_T of all states!
-

Calculating α

- At the start, α_1 is just given by starting probabilities multiplied by emission probabilities
 - E.g. $\alpha_1(\text{Late}) = P(\text{Late}) * P(\text{Happy}|\text{Late})$
 - Otherwise, recurse:
-

Forward Algorithm: Pseudocode

for all i from 1 through N :

```

initialize  $\alpha_1(q_i) = P(q_i)P(o_1|q_i)$ 
for all t from 2 through T:
  for all i from 1 through N:
    set  $\alpha_t(q_i)$  to the sum over all j of  $\alpha_{t-1}(q_j) P(q_i|q_j)P(o_t|q_i)$ 
return sum over all i of  $\alpha_T(q_i)$ 

```

Forward Algorithm Example

(Series of diagrams showing H S H observations)

Forward Algorithm Example

H S H

OT: $P(OT)P(H|OT) = 0.36$

L: $P(L)P(H|L) = 0.2$

VL: $P(VL)P(H|VL) = 0.08$

Forward Algorithm Example

OT: $0.36 \rightarrow *P(OT|OT)=0.8, *P(S|OT)=0.1$

L: $0.2 \rightarrow *P(OT|L)=0.4$

VL: $0.08 \rightarrow *P(OT|VL)=0.6$

$0.1*(0.36*0.8 + 0.2*0.4 + 0.08*0.6) = 0.0416$

Forward Algorithm Example

OT: 0.0416

L: 0.05

VL: 0.0744

Forward Algorithm Example

OT: $0.9*(0.0416*0.8 + 0.05*0.4 + 0.0744*0.6) = 0.0881$

L: 0.0182

VL: 0.0126

$P(\text{happy, sad, happy}) = 0.0881+0.0182+0.0126 = 0.119$

Decoding

- Forward algorithm tells us probability of observation sequence, regardless of underlying states
 - May want to know what states could have produced a given sequence
 - This is a task called decoding
 - Will use a dynamic programming approach again
-

The Viterbi Algorithm

- Dynamic programming approach to decoding a sequence of observations using an HMM
 - Again, work forward, looking at each possible state at each time step
 - We can use the same lattice approach
 - Define $v_t(q_i)$ as the probability that we're in state q_i at time t , assuming that we took the most probable path there
-

Calculating v

- At $t=1$, identical to forward algorithm
 - $v_1(q_i) = P(q_i)P(o_1|q_i)$
 - For all future time steps, look at work recursively—this time finding the previous state that maximizes the probability
 - $v_t(q_i) = \max_j (v_{t-1}(q_j)P(q_i|q_j)P(o_t|q_i))$
-

Pseudocode: Viterbi Algorithm

for all i from 1 through N :

 initialize $v_1(q_i) = P(q_i)P(o_1|q_i)$

for all t from 2 through T :

 for all i from 1 through N :

 set $v_t(q_i)$ to the maximum of all j of $v_{t-1}(q_j)P(q_i|q_j)P(o_t|q_i)$

 create a backpointer from $v_t(q_i)$ to the argmax

return the path traced back from the largest value of v_T

Viterbi Algorithm Example

(Series of diagrams showing H S H observations)

Viterbi Algorithm Example

H S H

OT: $P(OT)P(H|OT) = 0.36$

L: $P(L)P(H|L) = 0.2$

VL: $P(VL)P(H|VL) = 0.08$

Viterbi Algorithm Example

OT: 0.36

L: 0.2

VL: 0.08

$\max((0.36 \cdot 0.8 \cdot 0.1), (0.2 \cdot 0.4 \cdot 0.1), (0.08 \cdot 0.6 \cdot 0.1)) = 0.0288$

Viterbi Algorithm Example

OT: 0.0288

L: 0.02

VL: 0.048

Viterbi Algorithm Example

$\max((0.0288 \cdot 0.8 \cdot 0.9), (0.02 \cdot 0.4 \cdot 0.9), (0.048 \cdot 0.6 \cdot 0.9)) = 0.0259$

OT: 0.0259

L: 0.0072

VL: 0.0032

Example: Part of Speech Tagging

- Example: given a sentence, determine the part of speech of each word
- Need to account for words that can have different parts of speech depending on context
 - E.g. "set" can be a noun, a verb, or an adjective
- Knowing a word's POS can be helpful for many NLP tasks

HMMs for POS Tagging

- Each POS is a hidden label, and each word is an observation
- $P(\text{POS2} | \text{POS1})$ is the transition probability from POS1 to POS2
 - E.g. $P(\text{VB} | \text{NN})$ is the probability that our next word is a verb given that our current word is a noun
- $P(w | \text{POS})$ is the emission probability of w for POS
 - E.g. $P(\text{giraffe} | \text{NN})$ is the probability that an arbitrary noun is giraffe (probably fairly low!)

HMMs for POS Tagging

(Diagram reference)

Viterbi Algorithm: An Example

- Let's do an example with this sentence: "Janet will back the bill"
 - We'll be considering the following POS tags:
 - NNP (proper noun)
 - NN (common noun)
 - MD (modal)
 - VB (verb)
 - RB (adverb)
 - DT (determiner)
 - True labels: Janet/NNP will/MD back/VB the/DT bill/NN
-

Starting the Algorithm

- At the start, for a POS j , $v1(\text{pos}) = P(j) P(\text{Janet} | j)$
 - Calculating the initial probability of j and the emission probability of Janet from j
 - Here $P(\text{NNP}) = 0.28$ and $P(\text{Janet} | \text{NNP}) = 0.000032$, so $v1(\text{NNP}) = 0.000009$
 - For all other POS tags, the emission probability of Janet is 0, so we can fill in the lattice with 0
-

Word 2

- There was only one possibility for Janet, but there are 3 for will: MD, VB, and NN
 - In all cases, we will be multiplying by $v1(NNP)$, since all other paths back will be 0
 - $v2(MD) = v1(NNP) * P(MD|NNP) * P(will|MD) = 2.8e-8$
 - $v2(VB) = v1(NNP) * P(VB|NNP) * P(will|VB) = 2.5e-13$
 - $v2(NN) = v1(NN) * P(NN|NNP) * P(will|NN) = 1.1e-10$
-

Word 3

- For word 3 (back), we have four options: VB, NN, JJ, and RB
- For each of these options, we have multiple possible paths to trace back—will need to figure out which one is most likely
- Note that we only have to look one step back—even if we had multiple Janet options, we'd have already figured out the most likely one for each path
- We'll look at the options for VB (the correct answer)

Word 3

- To calculate $v3(VB)$, we again need to consider 3 possibilities
 - $v2(NN) * P(VB|NN) * P(back|VB) = 1.1e-10 * 0.0014 * 0.00067 = 1.0e-16$
 - $v2(VB) * P(VB|VB) * P(back|VB) = 2.5e-13 * 0.005 * 0.00067 = 8.4e-19$
 - $v2(MD) * P(VB|MD) * P(back|VB) = 2.8e-8 * 0.7968 * 0.00067 = 1.5e-11$
 - Note that MD is not just the best path here because it was a likely option for will, but also because it gave us a very high transition probability
-

Continuing On

- We could continue with the last two words, both of which are ambiguous
 - the can be a determiner or a proper name (due to names like The Hague)
 - In the last column, NN has a higher probability value than VB, so we trace its path back to get the right answer
-

Wrapping Up

- Hidden Markov Models can let us reason about sequences of observable events caused by unobservable states

- Several important tasks for HMMs: likelihood and decoding
- HMMs very useful when doing sequence labeling tasks on real data
 - E.g. label each word in a sentence with a POS tag